Linear Classifiers and Logistic Regression

4

Pre

CS 3244
Machine Learning

NUS National University of Singapore | Computing

Recap and Forecast

Naïve Bayes:

- Naïve: assumes independence between input features
- Bayes: estimates posterior from prior, marginals and likelihoods
- Smoothing caters for unseen data

k NN:

- Instance-based non-parametric
- Votes from nearest neighbor
- Larger k to smooth decision boundaries
- Problems with high dimensional data

Other important concepts for ML:

- Smoothing, Cost (Loss) Functions, Noisy Targets and the Curse of Dimensionality

Learning Outcomes for this week

- Describe the basic idea of linear classification;
- Investigate linear classification using the Perceptron learning algorithm (PLA);
- Understand how both linear and logistic regression works;
- Understand the mathematical proof on how PLA converges;

- Other important concepts:
  Gradient Descent and Non-linear mappings.

The Perceptron

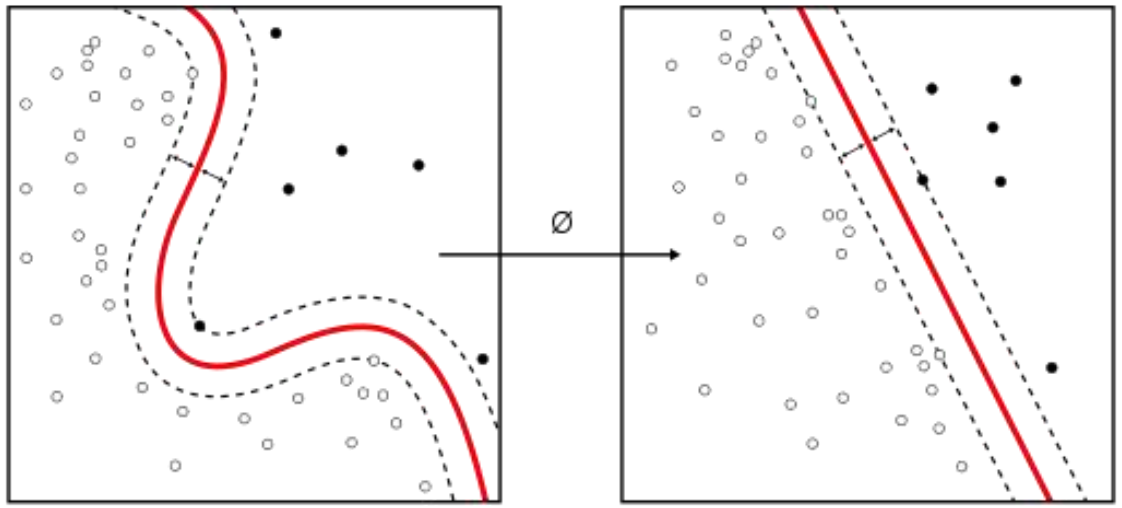
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

$\mathcal{H}$ for linear models

$\mathcal{H}$ represents the universe of possible hypotheses that a certain learning algorithm A can generate.

That is, given the choices of θ, what types of decision boundaries can it generate?



A ✗ B ✓

A simple hypothesis set – the perceptron

For input $\mathbf{x} = (x_1, x_2, \dots, x_n)$ *'attributes of a customer'*

- Approve credit if: $\sum_{i=1}^{n} \theta_i x_i > threshold$
- Deny credit otherwise: $\sum_{i=1}^{n} \theta_i x_i < threshold$

This linear formula $h \in \mathcal{H}$ can be written as:

$$h_\theta(x) = \text{sign}\left(\left(\sum_{i=1}^{n} \theta_i x_i\right) - threshold\right)$$

$$h(x) = \text{sign}((\sum_{i=1}^{n} \theta_i x_i) + \theta_0)$$

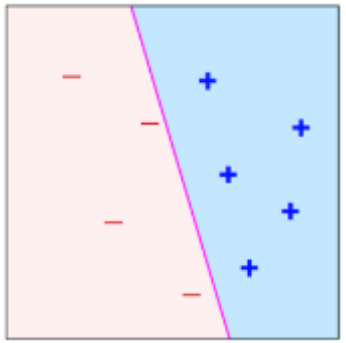
The 'bias weight' θ_0 corresponds to the threshold

Introduce an artificial coordinate $x_0 = 1$:

$$h_\theta(x) = \text{sign}(\sum_{i=0}^{n} \theta_i x_i)$$

In vector form, the perceptron implements

$$h(x) = \text{sign}(\theta^\top x)$$



Linearly separable data

A simple learning algorithm: PLA

The perceptron implements $h_\theta(\mathbf{x}) = \text{sign}(\theta^\top \mathbf{x})$

First initialize the weight vector θ in any fashion

Given the training set

$$\left((\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}) \ldots, (\mathbf{x}^{(m)}, y^{(m)})\right)$$

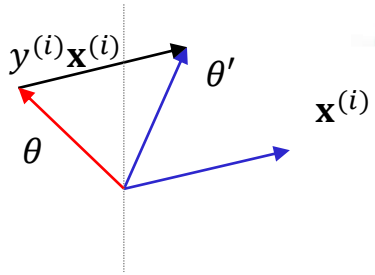
Pick a misclassified point $(\mathbf{x}^{(i)}, y^{(i)})$

$$\text{sign}(\theta^\top \mathbf{x}^{(i)}) \neq y^{(i)}$$

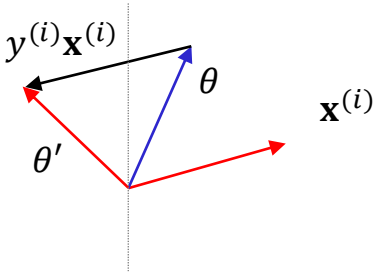
And update the weight vector

$$\theta' \leftarrow \theta + y^{(i)}\mathbf{x}^{(i)}$$

$y^{(i)} = +1$



$y^{(i)} = -1$



Iterations of PLA

One iteration of PLA:

$$\theta' \leftarrow \theta + y\mathbf{x}$$

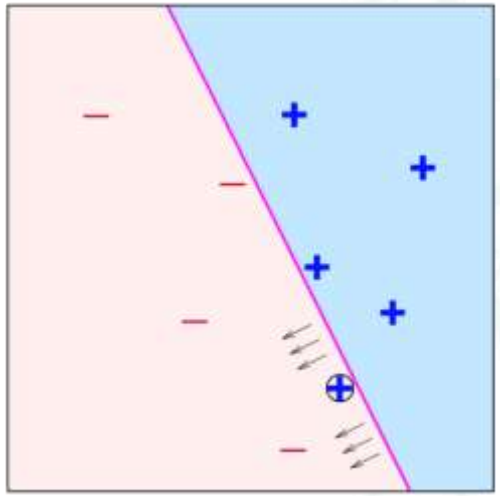
where $(\mathbf{x}, y)$ is a misclassified training point.

$t \leftarrow t + 1$

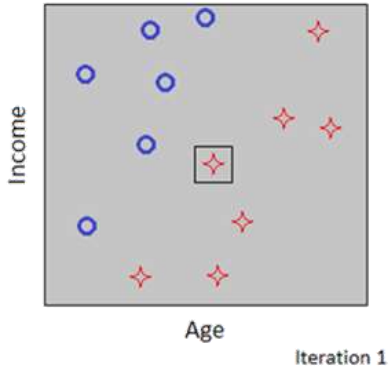
At iteration $t = 1, 2, 3, \ldots$, pick a misclassified point from:

$$\left(\left(\mathbf{x}^{(1)}, y^{(1)}\right), \left(\mathbf{x}^{(2)}, y^{(2)}\right) \ldots, \left(\mathbf{x}^{(m)}, y^{(m)}\right)\right)$$

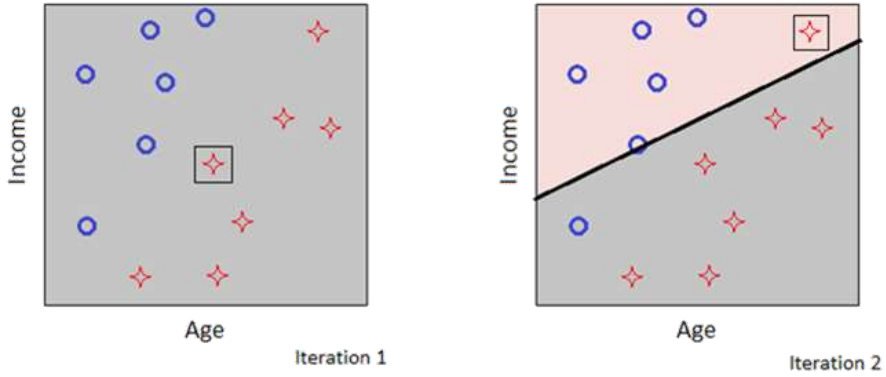
and run a PLA iteration on it.



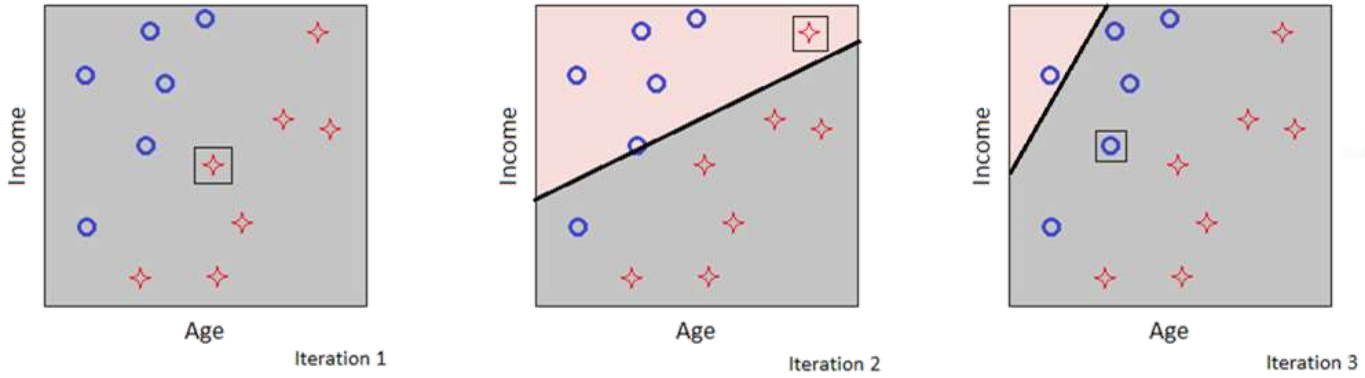
Does PLA work?



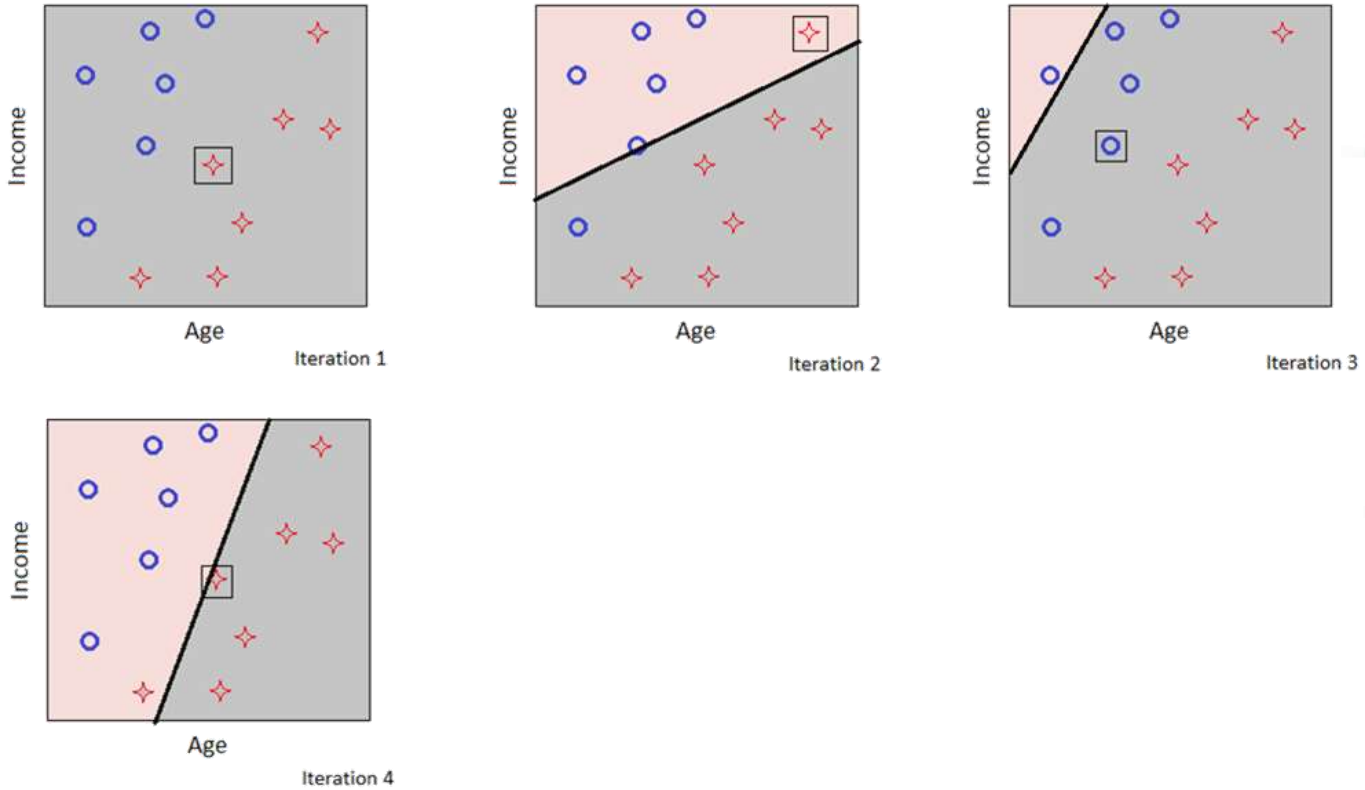
Does PLA work?



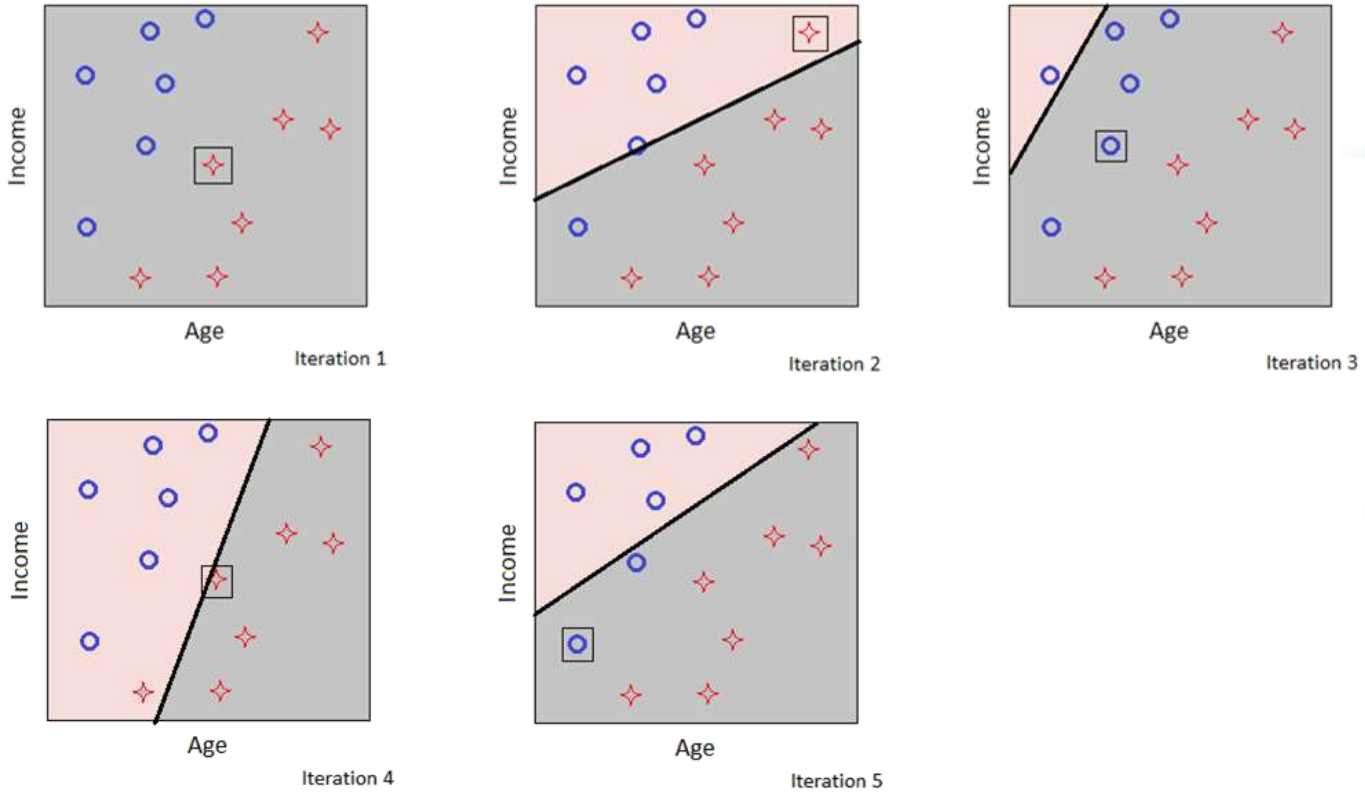
Does PLA work?



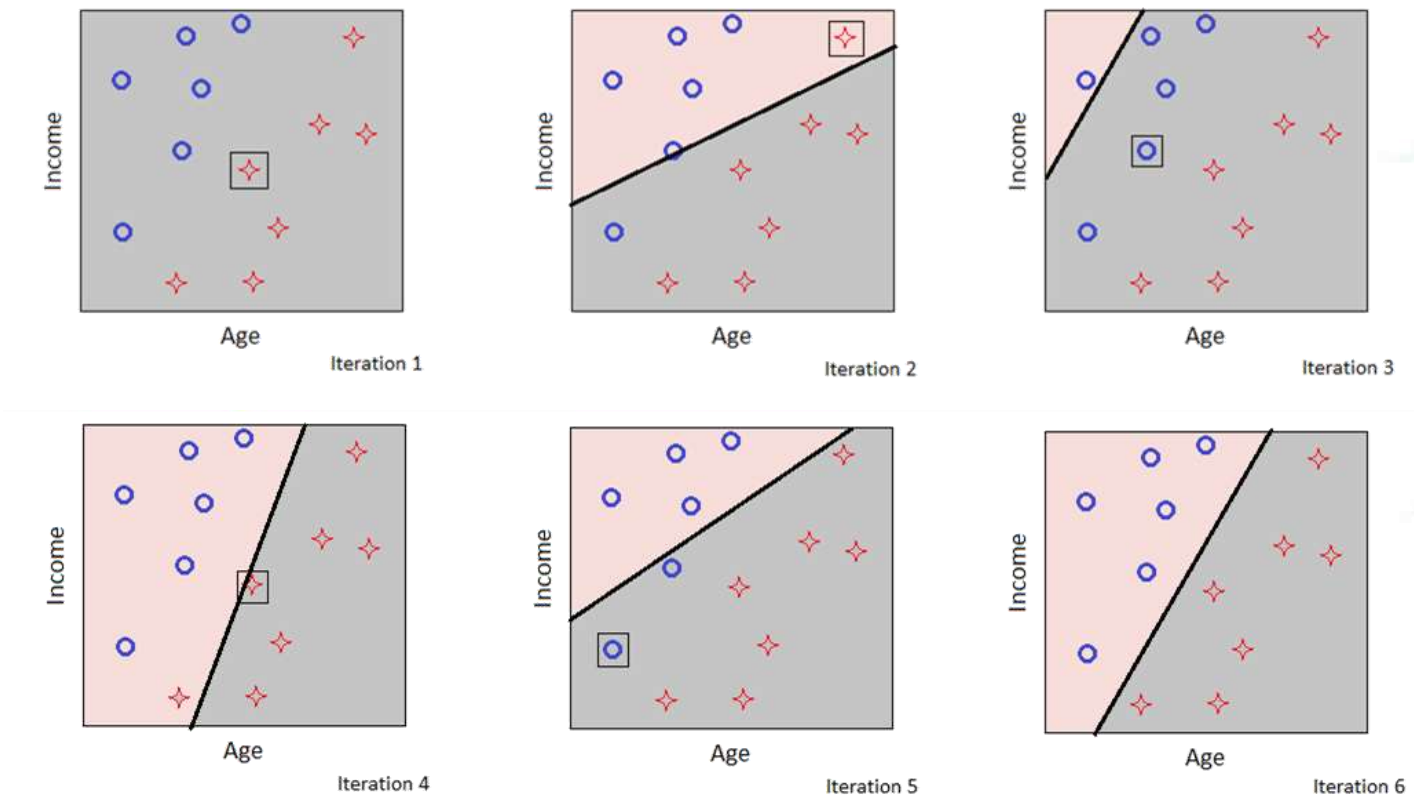
Does PLA work?



Does PLA work?

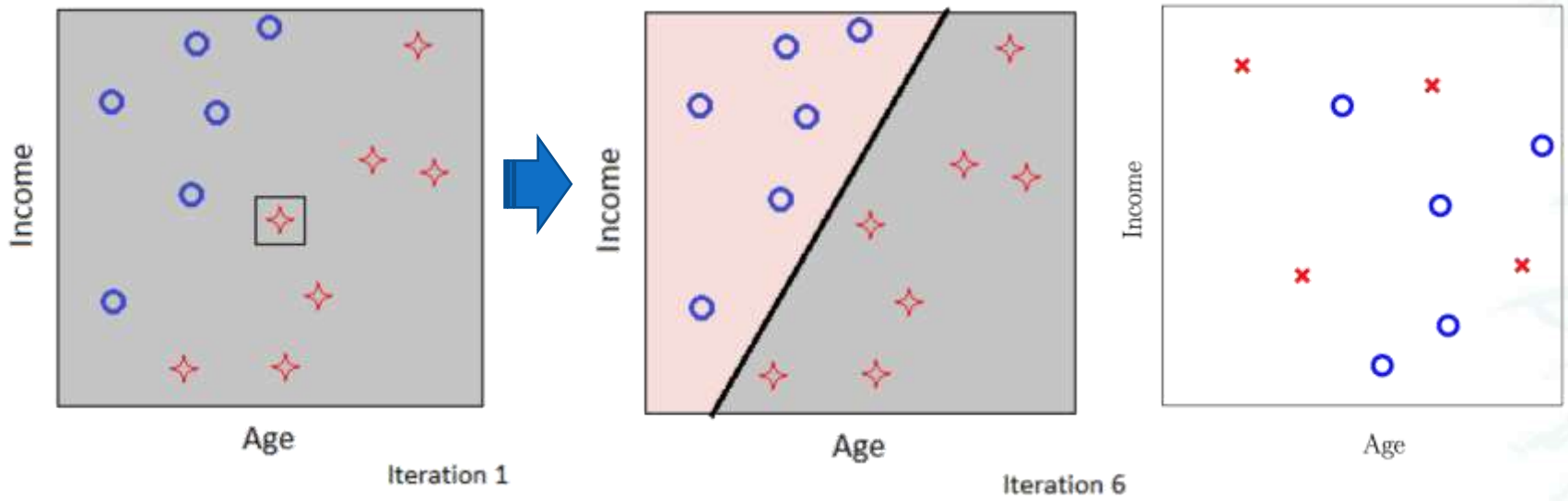


Does PLA work?



Does PLA work?

<u>Theorem</u>. If the data can be fit by a linear separator, then after some finite number of steps, PLA will find one.



Your Turn: but what if the perceptron cannot fit the data?

Summary – PLA

Perceptron Learning Algorithm

One of many iterative learning algorithms

Binary classification algorithm

Uses the signal $\theta^\top \mathbf{x}$

Starts with a random or zero weight vector θ

Nudges θ to correct individual errors on misclassified instances

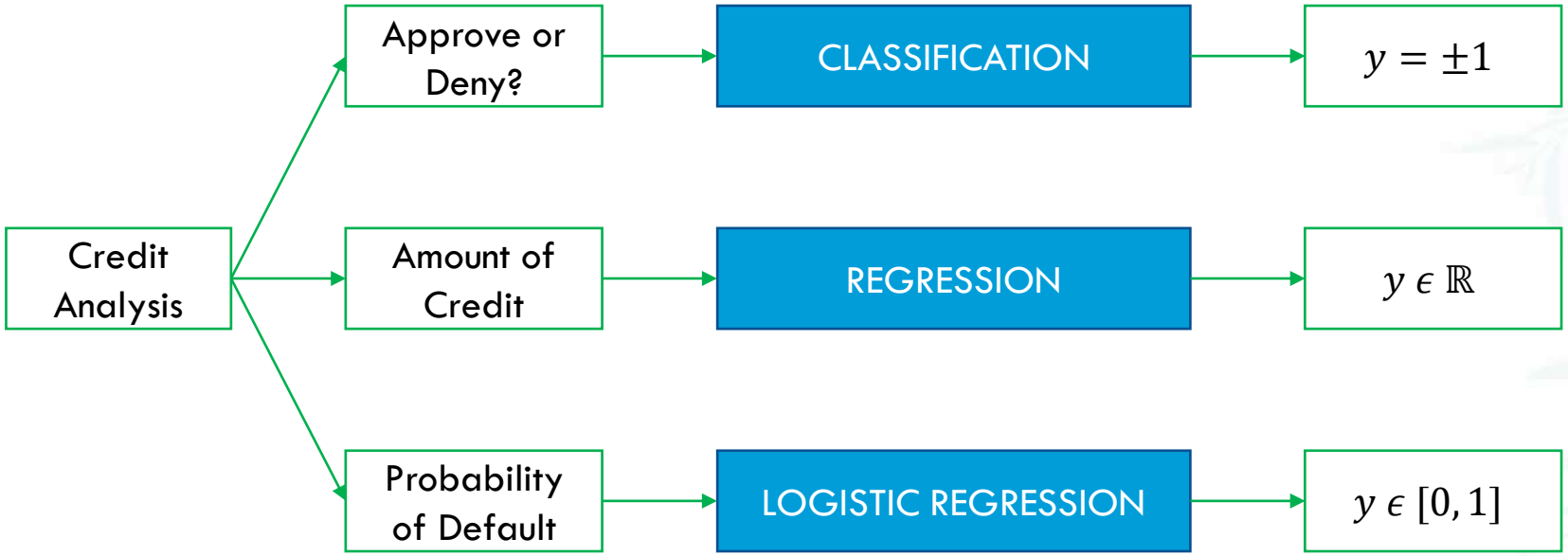
Converges given sufficient iterations (proof later)

Linear Models

CS3244 Machine Learning

NUS National University of Singapore | Department of Computer Science
School of Computing

Three learning problems



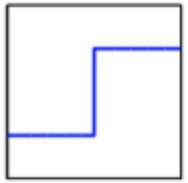
Linear models are the fundamental models.

The linear model is the first model to try.

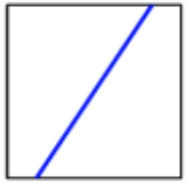
The linear signal

$$s = \theta^\top \mathbf{x}$$

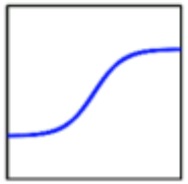
signal



$\text{sign}\,(\theta^\top \mathbf{x})$ $\quad y = \pm 1$

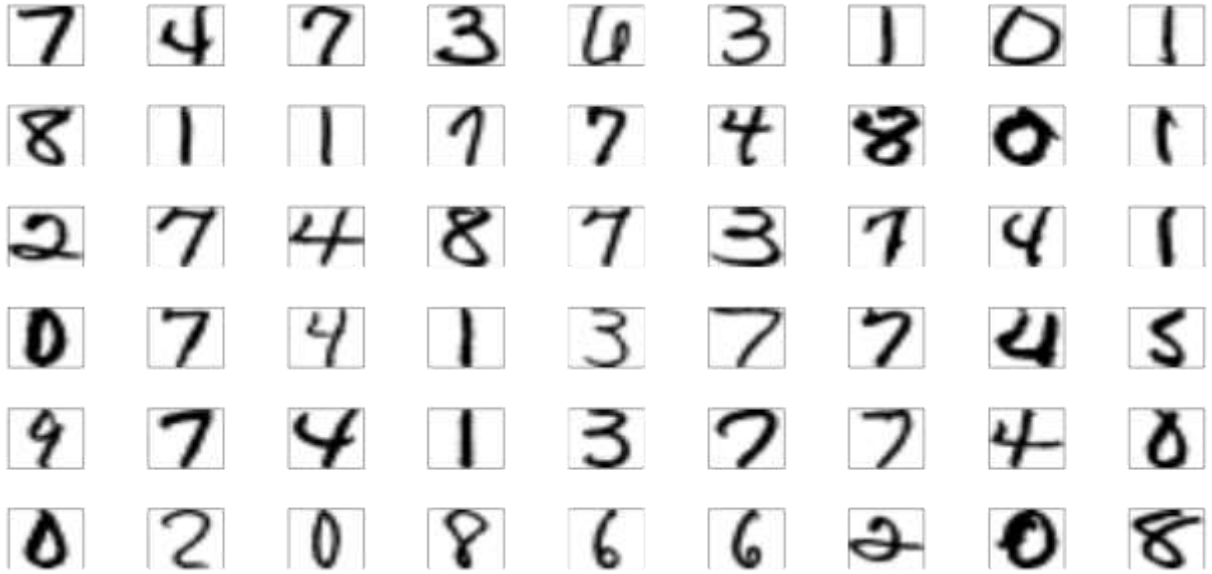


$\theta^\top \mathbf{x}$ $\quad y \in \mathbb{R}$ $\quad$ regression



$g(\theta^\top \mathbf{x})$ $\quad y \in [0, 1]$ $\quad$ probability

A real dataset



Each digit is a 16x16 pixel gray-intensity image.

[-1 -1 -1 -1 -1 -1 -1 -0.63 0.86 -0.17 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -0.99 0.3 1 0.31 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -0.41 1 0.99 -0.57 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -0.68 0.83 1 0.56 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -0.94 0.54 1 0.78 -0.72 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 0.1 1 0.92 -0.44 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -0.26 0.95 1 -0.16 -1 -1 -1 -0.99 -0.71 -0.83 -1 -1 -1 -1 -1 -0.8 0.91 1 0.3 -0.96 -1 -1 -0.55 0.49 1 0.88 0.09 -1 -1 -1 -1 0.28 1 0.88 -0.8 -1 -0.9 0.14 0.97 1 1 1 0.99 -0.74 -1 -1 -0.95 0.84 1 0.32 -1 -1 0.35 1 0.65 -0.10 -0.18 1 0.98 -0.72 -1 -1 -0.63 1 1 0.07 -0.92 0.11 0.96 0.30 -0.88 -1 -0.07 1 0.64 -0.99 -1 -1 -0.67 1 1 0.75 0.34 1 0.70 -0.94 -1 -1 0.54 1 0.02 -1 -1 -1 -0.90 0.79 1 1 1 1 0.53 0.18 0.81 0.83 0.97 0.86 -0.63 -1 -1 -1 -1 -0.45 0.82 1 1 1 1 1 1 1 1 0.13 -1 -1 -1 -1 -1 -1 -0.48 0.81 1 1 1 1 1 1 0.21 -0.94 -1 -1 -1 -1 -1 -1 -1 -0.97 -0.42 0.30 0.82 1 0.48 -0.47 -0.99 -1 -1 -1 -1]

Input representation

Raw input $\mathbf{x} = (x_0, x_1, x_2, x_3, x_4, \dots, x_{256})$

Too many (257) parameters!

Linear model: $(\theta_0, \theta_1, \theta_2, \dots, \theta_{256})$

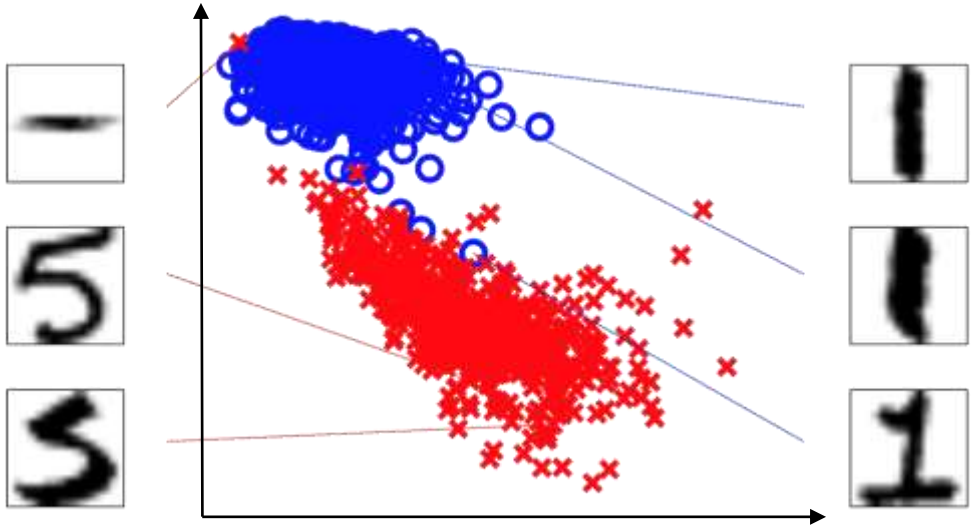
Features: extract useful information, e.g.,

Intensity and symmetry: $\mathbf{x} = (x_0, x_1, x_2)$

Linear model: $(\theta_0, \theta_1, \theta_2)$

Illustration of features

$\mathbf{x} = (x_0, x_1, x_2)$ $x_1 =$ intensity $x_2 =$ symmetry



Your Turn: which axes is which?

Pocket

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Iterations of PLA

One iteration of PLA:

$$\theta' \leftarrow \theta + y\mathbf{x}$$

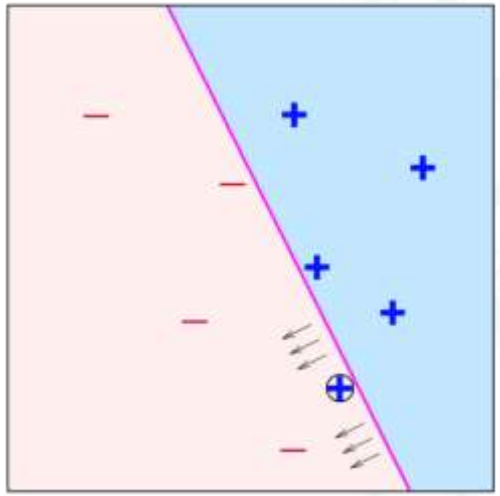
where $(\mathbf{x}, y)$ is a misclassified training point.

$t \leftarrow t + 1$

At iteration $t = 1, 2, 3, \ldots$, pick a misclassified point from:

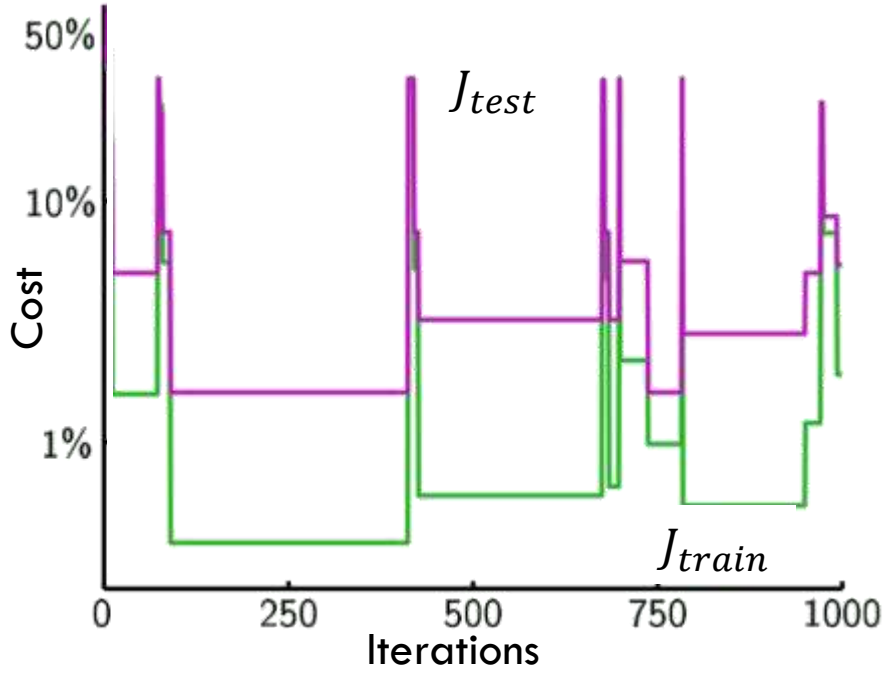
$$\left(\left(\mathbf{x}^{(1)}, y^{(1)}\right), \left(\mathbf{x}^{(2)}, y^{(2)}\right) \ldots, \left(\mathbf{x}^{(m)}, y^{(m)}\right)\right)$$

and run a PLA iteration on it.

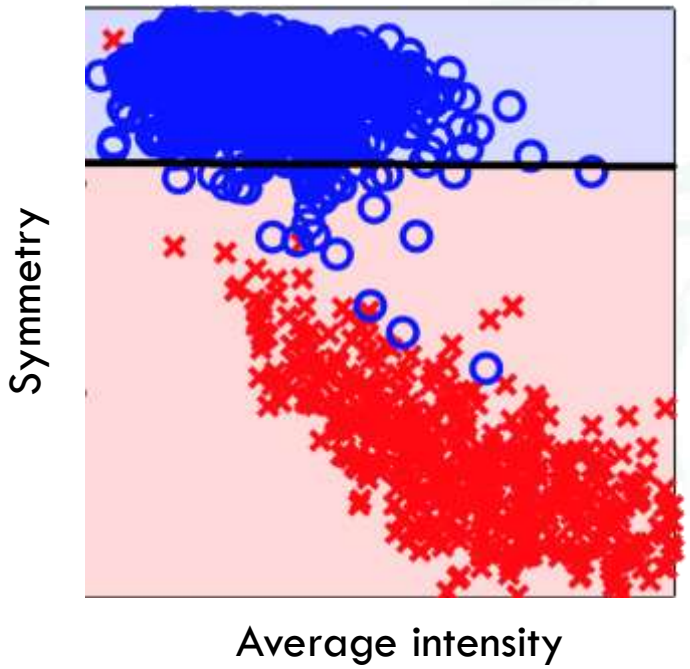


What PLA does

Evolution of J_{train} and J_{test}.

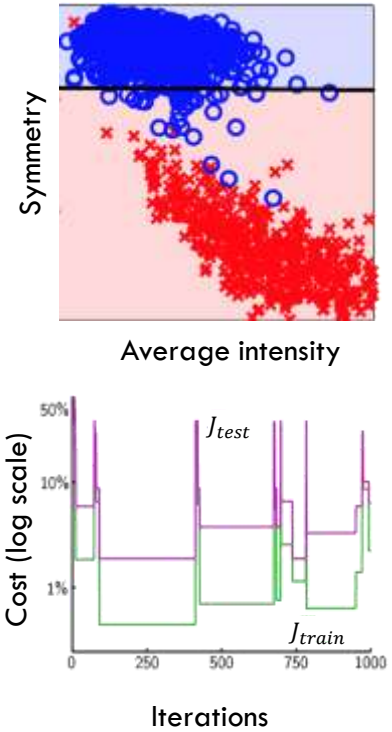


Final perceptron boundary



The pocket algorithm

PLA (for comparison):

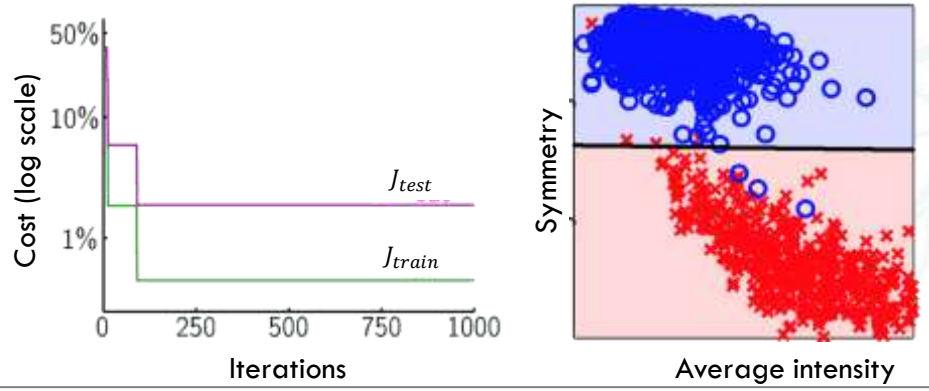


Pocket:

Run PLA

- At each step, keep the best J_{train} (and θ) so far

(it's not rocket science, but it works!)



Linear Regression

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Credit Approval

How much credit do we extend this person?

Criterion	Value
Age	32 years
Gender	Male
Salary	40 K
Debt	26 K
…	…
Years in Job	1 year
Years at Current Residence	3 years

~~Classification: Approve/Deny~~

- Regression: Credit line (real-valued dollar amount)
- Input: <*table on the left*>
- Linear regression output: $h_\theta(\mathbf{x}) = \sum_{i=0}^{n} \theta_i x_i = \theta^\top \mathbf{x}$
  $= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$
- Officers decide on credit lines based on historical data $\mathbf{X}$:
  $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})$

$y^{(j)} \in \mathbb{R}$ is the credit line for customer $x^{(j)}$;
regression tries to replicate this.

Artwork credits: Project DebtRelief

$\mathbb{R}$-valued cost function

How well does $h_\theta(\mathbf{x}) = \theta^\top \mathbf{x}$ approximate $f(\mathbf{x})$?

In linear regression, we use *squared error*: $(h_\theta(\mathbf{x}) - f(\mathbf{x}))^2$

Training error:

$\mathbb{R}$-valued cost function

How well does $h_\theta(\mathbf{x}) = \theta^\top \mathbf{x}$ approximate $f(\mathbf{x})$?

In linear regression, we use *squared error*: $(h_\theta(\mathbf{x}) - f(\mathbf{x}))^2$

Training error:

$$J_{train}(h) = \frac{1}{m}\sum_{j=1}^{m}\left(h_\theta\left(x^{(j)}\right) - y^{(j)}\right)^2$$

Average

How bad is our prediction?

Illustration of linear regression

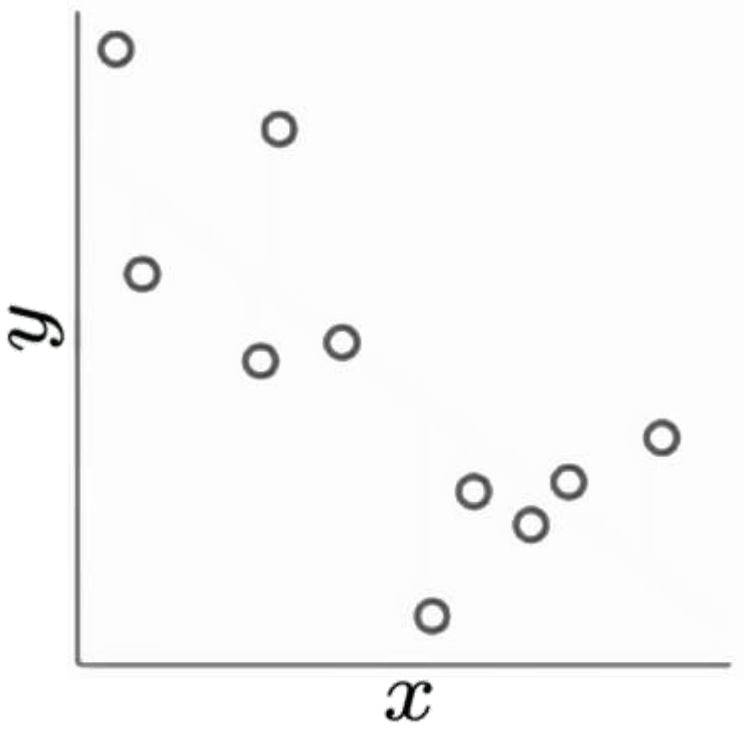
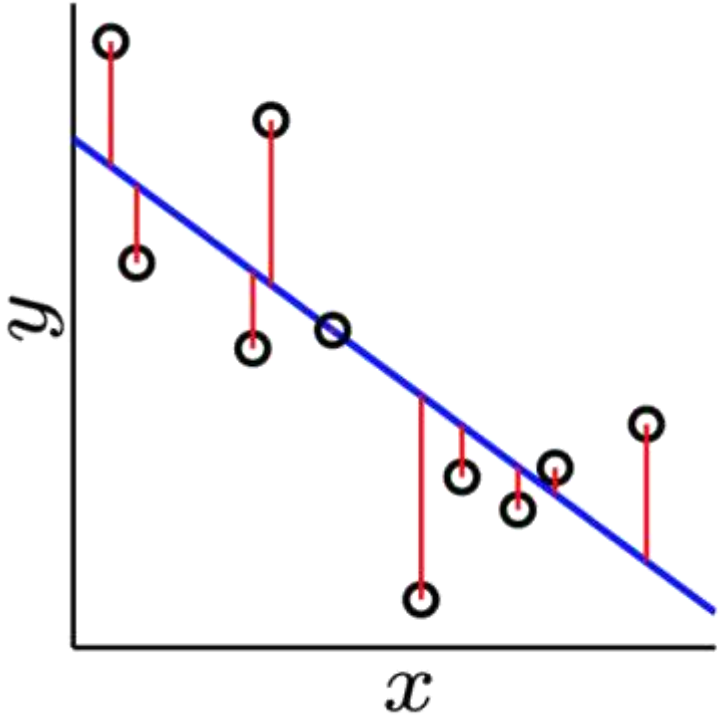
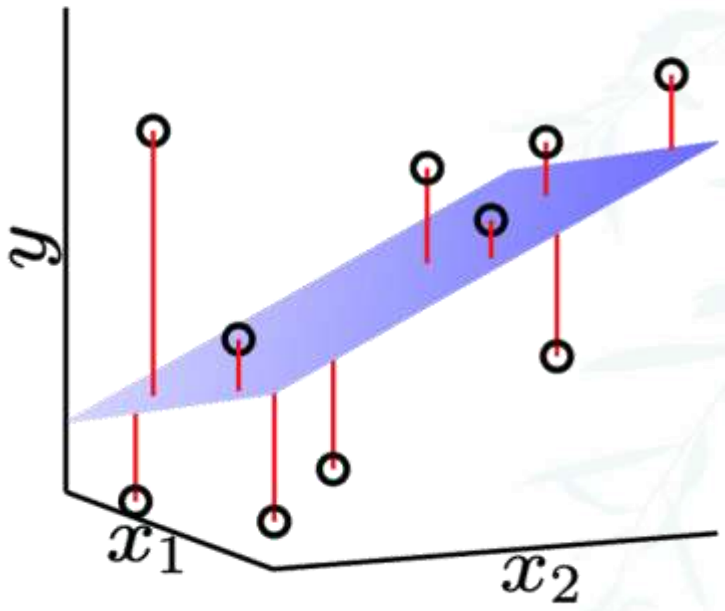


Illustration of linear regression





Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Logistic Regression

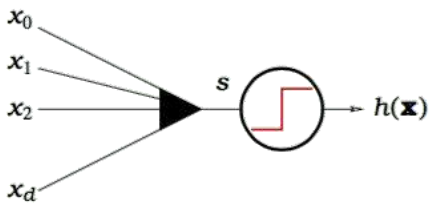
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

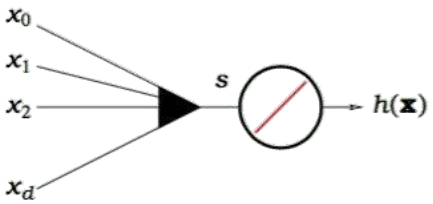
A third linear model

$$s = \sum_{i=0}^{n} \theta_i x_i$$

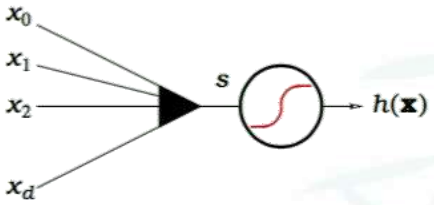
Linear classification



Linear regression



Logistic regression



The logistic function g

We choose a form for $g(\cdot)$ out of convenience.
We use the logistic function, which has the form:

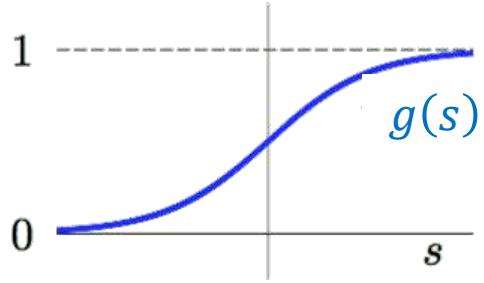
$$g(s) = \frac{exp^s}{1 + exp^s}$$

The logistic function g

We choose a form for $g(\cdot)$ out of convenience.
We use the logistic function, which has the form:

$$g(s) = \frac{exp^s}{1 + exp^s}$$

$g(s)$: needs to map any x: $\mathbb{R}$ to y: $[0,1]$



Soft threshold: uncertainty

Sigmoid: flattened out 's' shape

Probability Interpretation

$h_\theta(\mathbf{x}) = g(s)$ is interpreted as a probability

Logistic regression $\equiv y \in [0,1]$

Example: *Prediction of heart attacks*

Input $\mathbf{x}$: cholesterol level, age, weight, diabetic, etc.

output $g(s)$: Likelihood of a heart attack

The signal $s = \theta^\top \mathbf{x}$

"risk score"

Photo credits: Wikipedia - Chinese Buffet

Labels are binary, yet our outputs are probabilities

$$\mathbf{X} = \left(x^{(1)}, y^{(1)} = \pm 1\right), \ldots \left(x^{(m)}, y^{(m)} = \pm 1\right)$$

$x^{(j)}$ – a person's health information

$y^{(j)} = \pm 1$ – did they have a heart attack?

Why?

We cannot measure a <u>probability</u>.

We can only see the occurrence of an event and try to <u>infer</u> a probability.

Genuine Probability

Data $(\mathbf{x}, y)$ with binary y, generated by a noisy target:

$$P(y|x) = \begin{cases} f(\mathbf{x}) & \text{for } y = +1; \\ 1 - f(\mathbf{x}) & \text{for } y = -1. \end{cases}$$

The target $f: \mathbb{R}^n \to [0,1]$ is the probability

Learn $h_\theta(\mathbf{x}) = g(\theta^\top \mathbf{x}) \approx f(\mathbf{x})$

Cost Function for Logistic Regression

For logistic regression,

$$J_{train}(\theta) = \frac{1}{m}\sum_{j=1}^{m} ln(1 + exp^{-y^{(j)}\theta^{\top}x^{(j)}})$$

Iterative Solution

Compare to linear regression:

$$J_{train}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\left(\theta^{\top}x^{(j)} - y^{(j)}\right)^2$$

Closed-form Solution; Iteration also possible

Gradient Descent

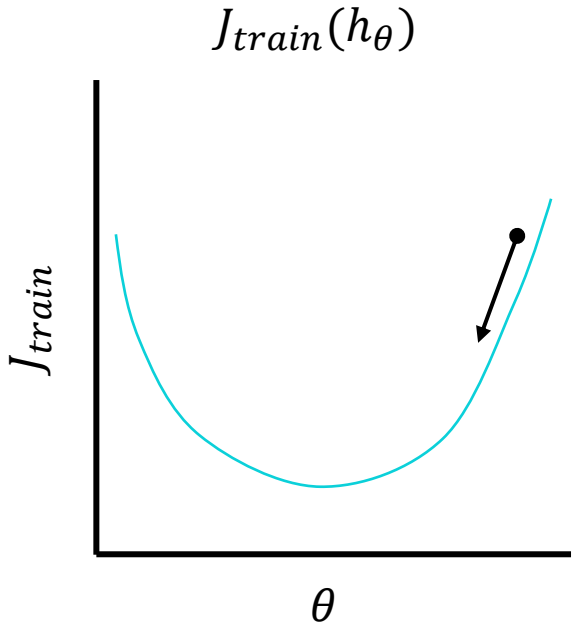
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Climbing up (down) one step at a time

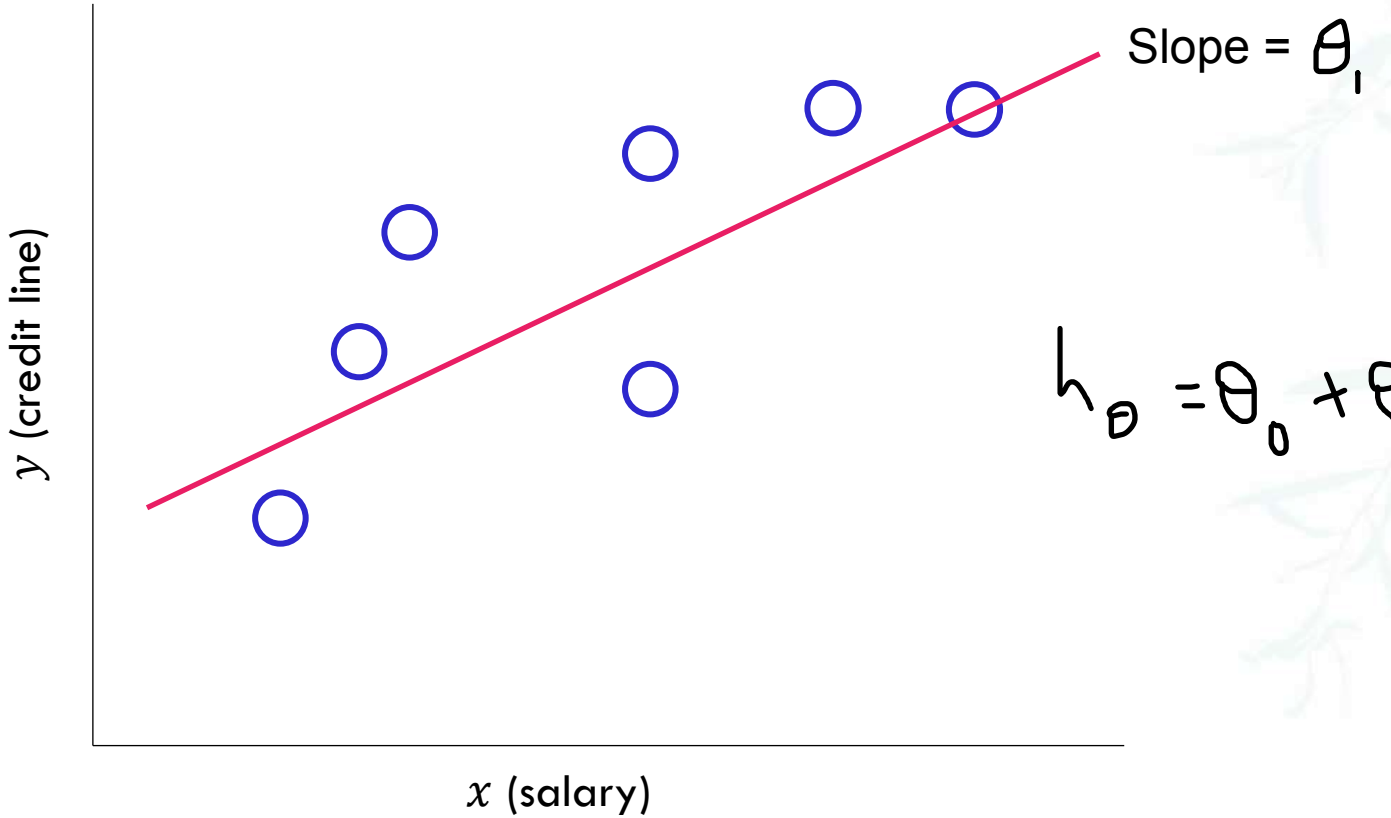
Photo credits: Wikimedia

Our J_{train} only has one valley



… Because J_{train} is a convex function of θ.

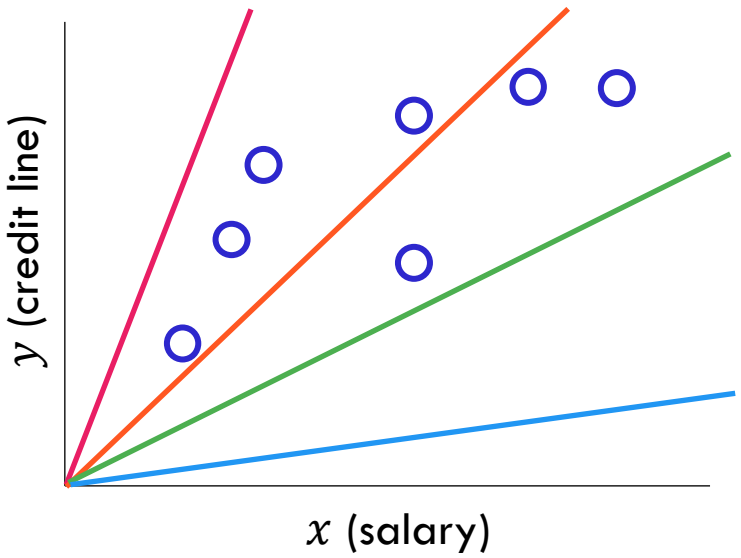
Univariate Linear Regression: Salary to predict Credit Line

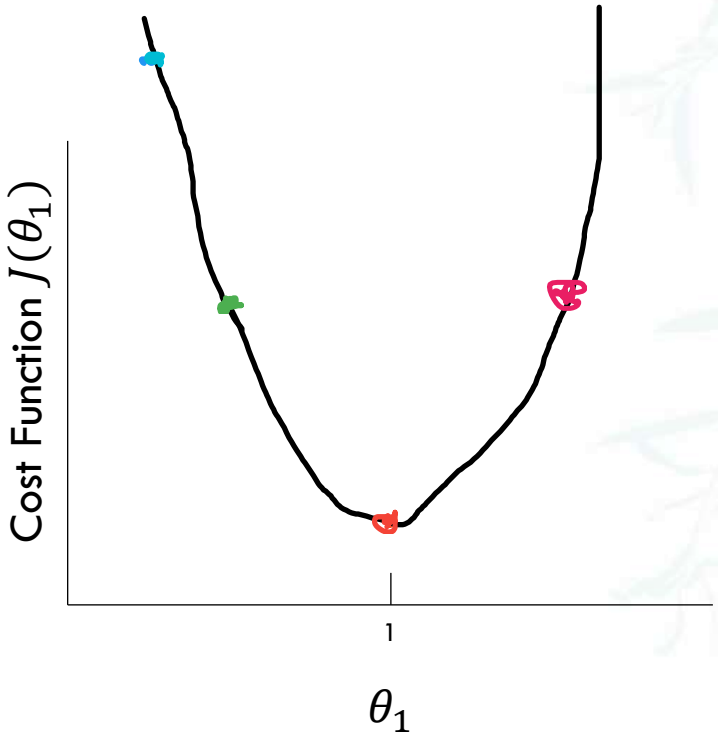


$$h_\theta = \theta_0 + \theta_1 x$$

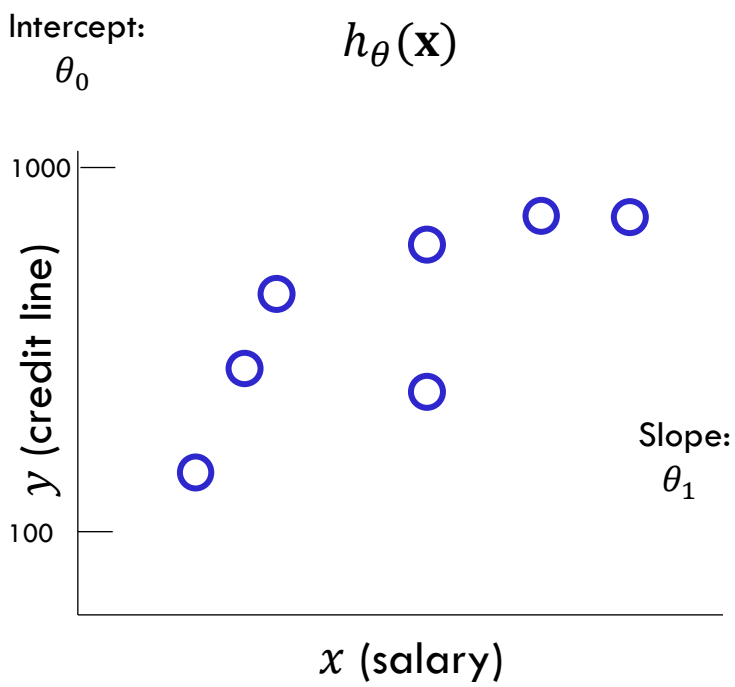
Ignoring the bias term

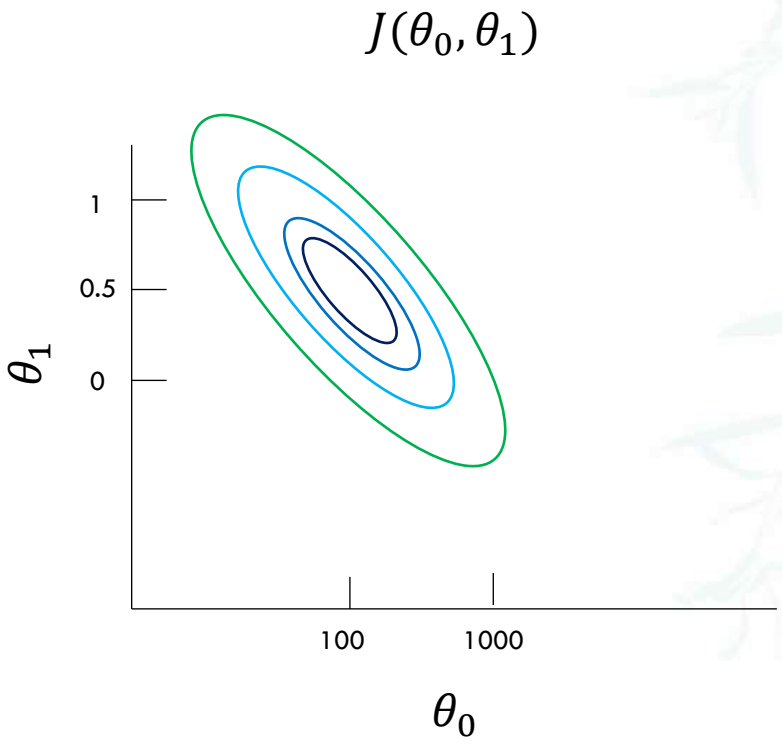
$h_\theta = \cancel{\theta_0} + \theta_1 x$





Cost Function $J(\theta_0, \theta_1)$





Iterative method: gradient descent

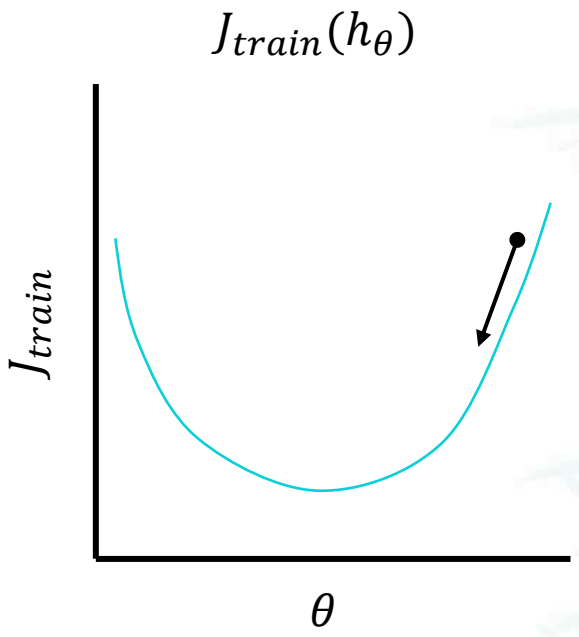
General method for nonlinear optimization

Start at $\theta(0)$; take a step along steepest slope

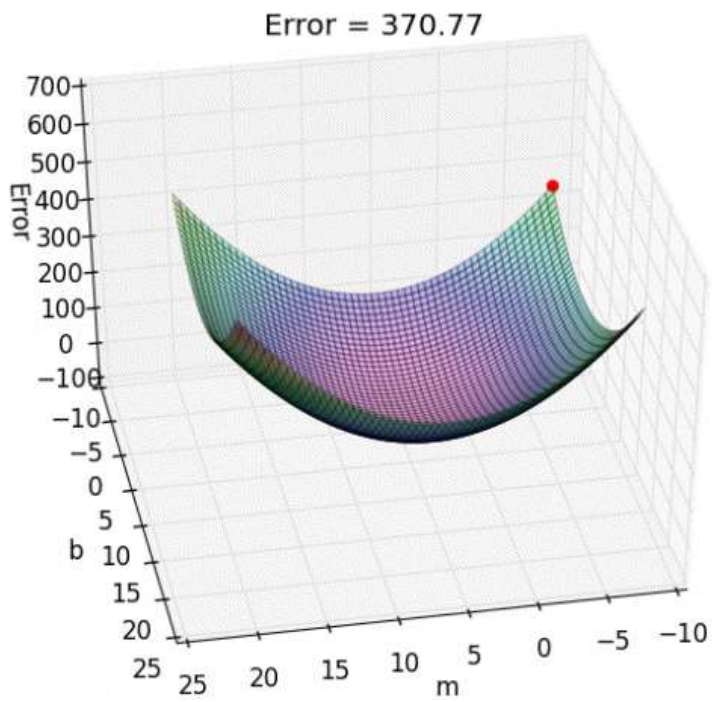
Fixed step size η : $\theta(1) = \theta(0) + \eta v$

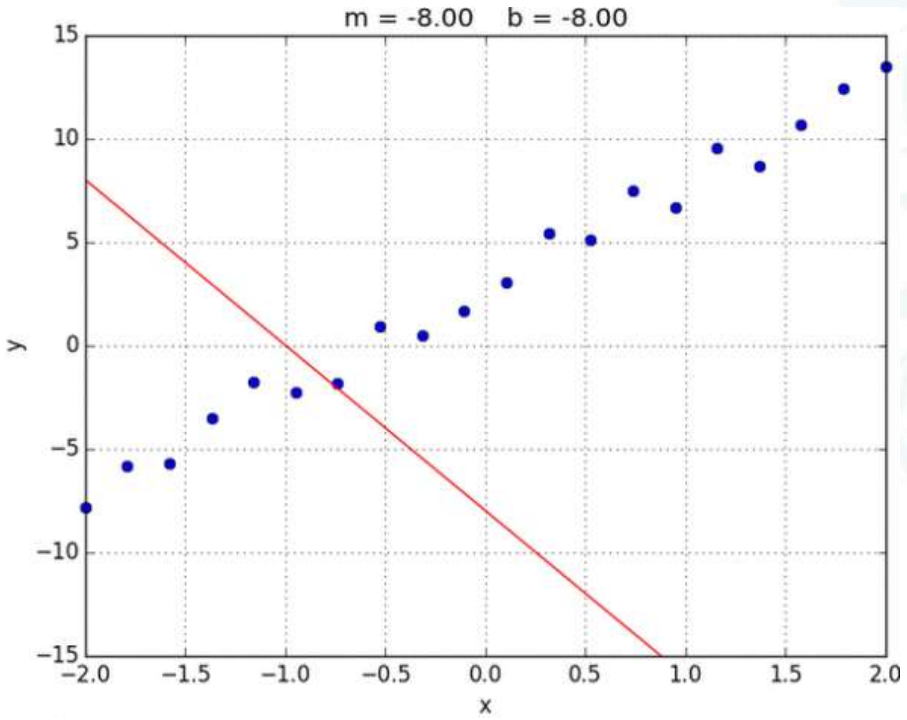
We get to pick the direction of v.
What is the best direction to pick?

The one to minimize J_{train}.



Gradient descent can minimize any smooth function.
Oh yeah!





Credits: Alykhan Tejani's Medium Post

Formula for the direction v

$$\Delta J_{train} = J_{train}\big(\theta(t+1)\big) - J_{train}\big(\theta(t)\big)$$

$$= J_{train}(\theta(t) + \eta v) - J_{train}\big(\theta(t)\big)$$

$$= \eta \nabla J_{train}(\theta(t)^{\top}) v \quad + O(\eta^2)$$

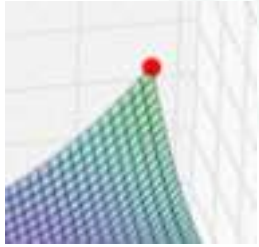
$$\geq -\eta \left\| \nabla J_{train}\big(\theta(t)\big) \right\|$$

Taylor approximation

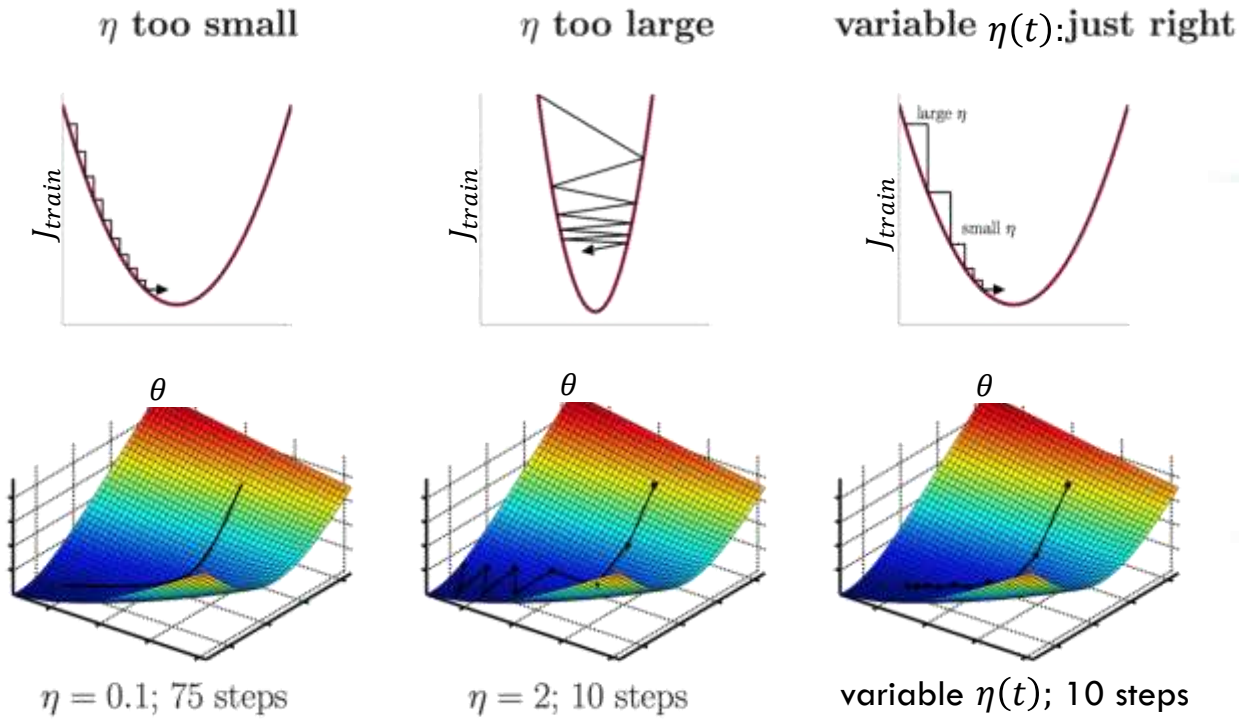
$f(x_1) = f(x_0) + \nabla f(x_0) \cdot (x_1 - x_0) + O(\|x_1 - x_0\|^2)$

Create v to be a unit vector:

$$v = -\frac{\nabla J_{train}(\theta(\mathrm{t}))}{\|\nabla J_{train}(\theta(t))\|}$$



Goldilocks step size



η should increase with the slope

Varying step size, for free

Instead of

$$\Delta\theta = \eta v$$

$$= -\eta \frac{J_{train}(\theta(t))}{\|J_{train}(\theta(t))\|}$$

Have

$$\Delta\theta = -\alpha J_{train}(\theta(t))$$

Fixed learning <u>rate</u> α instead of fixed learning <u>step</u> η

Logistic regression algorithm

1. Initialize the weights at $t = 0$ to $\theta(0)$
2. Do
3. Compute the gradient

$$\nabla(t) = \nabla J_{train}(\theta(t)) = -\frac{1}{m}\sum_{j=1}^{m}\frac{y^{(j)}x^{(j)}}{1 + e^{y^{(j)}\theta(t)^\top x^{(j)}}}$$

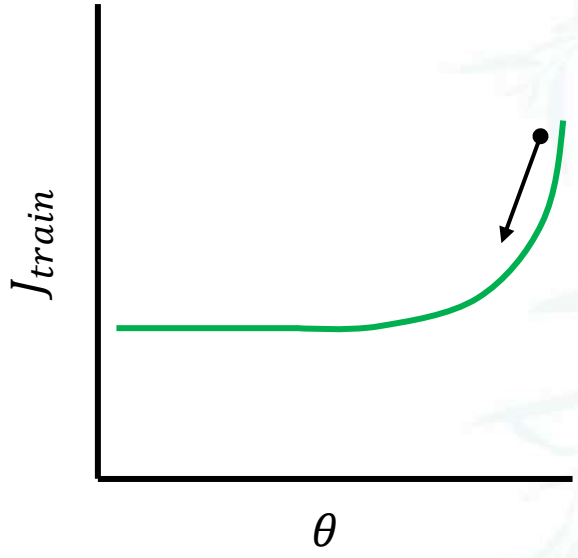
4. // Move in the direction $v(t) = -\nabla(t)$
   Update the weights $\theta(t+1) = \theta(t) - \alpha\nabla J_{train}$
5. Continue to next iteration, until it is <u>time to stop</u>
6. Return the final weights θ^*

Termination Condition

When to stop?

Natural choice: *gradient* < *threshold*

But lots of flat regions in most spaces:



Instead, use criteria:

1. error change is small and/or;
2. error is small;
3. maximum number of iterations is reached.